

RedeApoio

Rotinas de Manutenção e Correção

Para programadores

Documentação técnica e operacional do sistema RedeApoio

Gerado em 04 de agosto de 2026

Origem: docs/05-rotinas-de-manutencao.md

Este PDF é gerado automaticamente a partir da documentação do repositório. A versão sempre atual está em **docs/05-rotinas-de-manutencao.md**, no GitHub — em caso de divergência, vale o que está lá.

5 — Rotinas de manutenção e correção

Como diagnosticar, corrigir e publicar mudanças sem quebrar o que já funciona. Escrito tanto para quem programa quanto para um agente de IA que assuma o projeto.

Antes de qualquer alteração

1. **Leia Referência Técnica** — em especial a seção "As invariantes que não podem ser quebradas". A maioria dos bugs graves deste projeto veio de violar uma delas.
2. **Faça backup do banco**, se for mexer em produção: `bash scripts/backup-db.sh`
3. **Trabalhe na branch 1.0**. É a branch de produção. A `main` está desatualizada e contém a versão antiga (Vercel/Supabase).

Ciclo de correção de um bug

1. Reproduzir

Não corrija por dedução. Reproduza — e **no mesmo ambiente do relato**. O bug do campo de login vazio (`41b2acb`) era invisível no desktop: só acontecia em teclado de celular, por causa do modo de composição do Gboard.

Perguntas que encurtam o diagnóstico:

- Celular ou computador? Android ou iPhone? Qual navegador?
- Qual perfil estava logado (candidato, liderança, apoiador, admin)?
- Aconteceu uma vez ou sempre acontece?
- Apareceu alguma mensagem de erro? Peça print ou vídeo

2. Localizar

```
# Erro visível no navegador → frontend
grep -n "trecho da mensagem" frontend/index.html

# Erro de API (4xx/5xx) → backend
grep -rn "trecho da mensagem" backend/src/routes/

# Erro 500 sem mensagem → logs do servidor
docker service logs --tail 200 redeapoio_redeapoio-app
```

3. Entender antes de mudar

Este código tem comentários densos que explicam **por que** cada decisão estranha existe. Muitos marcam correções de bugs reais. Leia o comentário antes de "simplificar" — a linha esquisita costuma ser a correção.

Exemplos: `types.setTypeParser(1082, ...)` em `db.js`; o `id` explícito ao inserir na tabela `apoiaadores`; `asyncHandler` em toda rota.

4. Testar localmente

Não há ambiente de homologação. Para subir uma cópia local (precisa de Docker):

```
# Postgres descartável
docker run -d --name pg-teste \
  -e POSTGRES_DB=redeapoio -e POSTGRES_USER=redeapoio -e POSTGRES_PASSWORD=teste \
  -p 5432:5432 postgres:16-alpine

# Backend apontando para ele
cd backend && npm install
PGHOST=localhost PGPORT=5432 PGUSER=redeapoio PGPASSWORD=teste PGDATABASE=redeapoio \
JWT_SECRET=segredo-de-teste PUBLIC_URL=http://localhost:3000 \
FRONTEND_DIR=../frontend node src/server.js
```

Acesse `http://localhost:3000`. Crie um admin:

```
node -e "require('bcryptjs').hash('teste1234',10).then(console.log)"
docker exec -it pg-teste psql -U redeapoio -d redeapoio -c \
  "INSERT INTO usuarios (nome, login, senha_hash, perfil)
  VALUES ('Admin','admin','<COLE_O_HASH>','admin');"

```

Ao terminar: `docker rm -f pg-teste`.

Para testar comportamento de **celular**, use o Chrome no computador (F12 → ícone de dispositivo) — mas saiba que **o modo dispositivo não simula composição de teclado**. Bug de digitação em campo com máscara só se confirma em celular de verdade, ou pelo `chrome://inspect` com o aparelho conectado.

5. Verificar a sintaxe do frontend

O `index.html` não tem build, então um erro de sintaxe no JavaScript só apareceria no navegador do usuário. Antes de comitar:

```
node -e "
const fs=require('fs');
const h=fs.readFileSync('frontend/index.html','utf8');
[...h.matchAll(/<script>([\s\S]*?)<\script>/g)].forEach((s,i)=>{
  try{ new Function(s[1]); console.log('script #' + i + ' OK'); }
  catch(e){ console.log('script #' + i + ' ERRO: ' + e.message); process.exit(1); }
});
"
```

6. Commitar

Mensagens em português, no imperativo, dizendo **o que mudou e por quê** — é o padrão de todo o histórico:

```
Corrige campo de login que ficava vazio ao digitar pelo celular

O oninput reescrevia this.value a cada tecla. No celular o teclado digita
em modo composicao e reescrever o value nesse meio-tempo faz o teclado
descartar tudo. No desktop nao acontecia porque teclado fisico nao usa
composicao.
```

7. Publicar

```
ssh root@IP_DO_SERVIDOR
cd /opt/redeapoiopolitico
bash scripts/backup-db.sh
git pull origin 1.0
bash build.sh
docker service logs --tail 50 redeapoio_redeapoio-app
```

Teste no navegador logo em seguida. Para voltar atrás:

```
git log --oneline -5
git checkout <commit-anterior>
bash build.sh
```

Roteiro de teste manual

Não há testes automatizados. Rode este roteiro depois de qualquer mudança que toque autenticação, pirâmide ou cadastro.

Autenticação

- ☐ Login como admin, candidato, liderança e apoiador
- ☐ Login usando **e-mail** no lugar do usuário
- ☐ Senha errada mostra "Usuário ou senha incorretos"
- ☐ Recarregar a página mantém a sessão
- ☐ Sair e voltar exige login de novo

Pirâmide

- ☐ Rede de Apoio mostra os níveis com a contagem certa de indicados
- ☐ Liderança vê só a própria rede
- ☐ Candidato vê a rede inteira
- ☐ Reorganizar hierarquia (mudar responsável) funciona
- ☐ Tentar pendurar alguém sob um descendente dele é bloqueado
- ☐ Limite de indicações é respeitado

Cadastro

- ☐ Cadastrar apoiador pelo painel
- ☐ Link pessoal: quem se cadastra entra um nível abaixo, no responsável certo
- ☐ Link por nível: entra sem responsável, no nível escolhido
- ☐ Telefone/título duplicado na mesma rede é bloqueado
- ☐ Título de eleitor inválido é rejeitado
- ☐ **Pelo celular:** todos os campos aceitam digitação, incluindo login
- ☐ **Pelo celular:** data de nascimento aceita digitação (sem calendário)

Contas

- ☐ Senha temporária força troca no primeiro acesso
- ☐ Termo LGPD aparece e o aceite grava em `termos_aceite`
- ☐ "Esqueci minha senha" envia o e-mail e o link funciona
- ☐ Candidato redefine senha de liderança
- ☐ Liderança redefine senha de apoiador da própria rede

Admin

- ☐ Central de Vagas lista os candidatos
- ☐ Criar candidato mostra a senha temporária
- ☐ Abrir workspace de um candidato mostra os dados dele
- ☐ Data de desativação no passado bloqueia o candidato **e a rede dele**

Diagnóstico por sintoma

"0 indicados" para todo mundo na pirâmide

Quase certamente a invariante do id-espelho foi violada (ver [Referência Técnica](#)). Confirme:

```
-- Deve retornar ZERO linhas. Cada linha é um usuário sem ficha correspondente.
SELECT u.id, u.nome, u.perfil FROM usuarios u
WHERE u.perfil IN ('lideranca','apoiador')
AND NOT EXISTS (SELECT 1 FROM apoiadores a WHERE a.id = u.id);
```

Se retornar linhas, reiniciar o app já resolve — `001_init.sql` tem o reparo (linhas 86-94) e roda a cada boot:

```
docker service update --force redeapoio_redeapoio-app
```

Requisição fica "carregando" para sempre

Rota async sem `asyncHandler`. Procure:

```
grep -n "router\\.\\(get\\|post\\|put\\|delete\\)(\\.\\*async" backend/src/routes/*.js | grep -v asyncHandler
```

Campo do celular não aceita digitação

Algun handler `oninput` está reescrevendo `input.value` durante a composição do teclado. Procure e aplique o padrão do `mascaraLogin()`:

```
grep -n "oninput=\"this.value" frontend/index.html
```

500 sem mensagem clara

```
docker service logs --tail 300 redeapoio_redeapoio-app | grep -A 20 Error
```

Erro de coluna inexistente (`column X does not exist`) significa que o código referencia uma coluna que falta no schema. A correção vai em `001_init.sql`, como `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`, e sobe com `build.sh` — foi exatamente assim que `usuarios.estado` foi resolvido no commit `14c0f04`.

Banco crescendo demais

```
docker exec $(docker ps -q -f name=redeapoio_redeapoio-postgres) \
psql -U redeapoio -d redeapoio -c "
SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) AS tamanho
FROM pg_catalog.pg_statio_user_tables ORDER BY pg_total_relation_size(relid) DESC;"
```

`termos_aceite` cresce sozinha por design (uma linha por aceite) e **não deve ser limpa** — é a prova de consentimento.

Como adicionar um campo novo ao cadastro

Roteiro completo, na ordem certa:

1. **Schema** — no fim de `backend/migrations/001_init.sql`:

```
ALTER TABLE apoiadores ADD COLUMN IF NOT EXISTS meu_campo TEXT;
ALTER TABLE usuarios ADD COLUMN IF NOT EXISTS meu_campo TEXT;
```

Adicione nas **duas** tabelas se o campo também vale para quem tem login.

2. **Backend** — inclua o campo no `req.body` desestruturado, nos `INSERT` e nos `UPDATE` de: `routes/apoiadores.js`, `routes/usuarios.js` e `routes/public.js` (autocadastro). Não esqueça a sincronização da ficha-espelho em `usuarios.js` (`UPDATE apoiadores SET ...`).
3. **Frontend** — adicione o `<input>` nos formulários correspondentes: autocadastro (~linha 284), cadastrar apoiador (`renderCadastrar`), editar apoiador (`editarApoiador`), novo usuário e editar usuário. E inclua o campo no envio de cada `salvar*()`.
4. **Exportação** — confira se aparece no CSV (`exportCSV`).
5. **Teste** com o roteiro acima, sem esquecer o celular.

Contexto para um agente de IA que assumir o projeto

Se você é um Claude (ou outro agente) entrando neste repositório agora:

Leia nesta ordem: `CLAUDE.md` (raiz) → `docs/04-referencia-tecnica.md` → este arquivo. Só depois abra o código.

O que mais importa:

1. **Sempre reproduza antes de corrigir.** Vários bugs deste projeto só aparecem em contextos específicos (celular, um perfil só, uma profundidade de pirâmide). Corrigir por dedução gera regressão.
2. **Os comentários do código são a memória do projeto.** Quase todo trecho estranho é a correção de um bug real e tem o motivo escrito ao lado. Leia antes de refatorar.
3. `001_init.sql` **roda em todo boot.** Qualquer instrução nova precisa ser idempotente. Nunca destrutiva.
4. **A invariante** `apoiadores.id == usuarios.id` é a coisa mais fácil de quebrar e a de consequência mais visível.
5. **Não "modernize" o frontend.** Arquivo único, JS puro, `onclick` inline — é escolha deliberada. Introduzir build quebra o fluxo de deploy inteiro.

6. **Comentários e mensagens de commit em português**, explicando o porquê.
7. **Não invente estrutura nova**. Rota nova entra no arquivo de rotas existente, tela nova entra no `index.html`.
8. **Antes de mexer em produção, backup**. Sempre.